

ĐỒ ÁN TỐT NGHIỆP · ĐỀ TÀI SỐ 4

LearnQuiz

Nền tảng học tập và quiz trực tuyến

Học viên thực hiện:

VŨ TÂM THIÊN TÍN

Giảng viên hướng dẫn:

NGUYỄN LÊ HOÀNG THÔNG

Nội dung trình bày

01

Bài toán và phạm vi

Ba vai trò, thang điểm 4 – 4 – 2

02

Kiến trúc và cơ sở dữ liệu

Ba tầng · 11 bảng · 9 ràng buộc toàn vẹn

03

Ba quyết định kỹ thuật cốt lõi

Không rõ ràng đáp án · chấm ở máy chủ · máy trạng thái

04

Tích hợp AI

Ba tính năng Gemini và nguyên tắc kiểm duyệt

05

Kiểm thử và triển khai

357 phép kiểm · Docker · CI/CD · Render + Vercel

06

Demo trực tiếp

8 phút đi qua đủ ba vai trò

Bài toán và mục tiêu

Nhu cầu thực tế

- Nền tảng thương mại thừa tính năng: thanh toán, tiếp thị, quy mô lớn.
- Trung tâm đào tạo nhỏ chỉ cần ba việc: soạn nội dung, học và tự đánh giá, kiểm soát chất lượng trước khi công bố.
- Đề tài đủ nhỏ để hoàn thành, đủ phức tạp để chạm vào những vấn đề thật của hệ thống nhiều người dùng.

Mục tiêu của đồ án

- Hệ thống hoàn chỉnh, chạy thật trên Internet
- Đủ chức năng của ba vai trò theo đề tài
- Ràng buộc toàn vẹn đặt ở tầng cơ sở dữ liệu
- Ba tính năng AI có kiểm duyệt đầu ra
- Kiểm thử tự động chạy được trong CI
- Đóng gói Docker và triển khai lên đám mây

Phạm vi: ba vai trò, thang điểm 4 – 4 – 2

Học viên

4,0

- Tìm và lọc khóa học
- Ghi danh khóa miễn phí
- Học theo lộ trình
- Làm quiz, xem điểm, làm lại
- Theo dõi tiến độ
- AI giải thích đáp án sai

Giảng viên

4,0

- Soạn khóa học và bài học
- Sắp xếp thứ tự bài
- Soạn quiz 4 đáp án
- AI sinh câu hỏi
- Gửi khóa học chờ duyệt
- Thống kê lớp học

Quản trị viên

2,0

- Duyệt / từ chối khóa học
- Gỡ khóa đang công khai
- Quản lý danh mục
- Khóa tài khoản vi phạm
- Thống kê toàn hệ thống

Công nghệ và lý do lựa chọn

1 TypeScript

Cả hai đầu — bắt lỗi kiểu lúc biên dịch, dùng chung kiểu dữ liệu API

2 Node.js + Express

Chuỗi middleware rõ ràng: bảo vệ trước, nghiệp vụ sau

3 Prisma + PostgreSQL

Lược đồ là nguồn sự thật; `select` chặn rò rỉ dữ liệu ngay tại truy vấn

4 React 19 + Vite + MUI

Ứng dụng một trang, đáp ứng nhiều kích thước màn hình

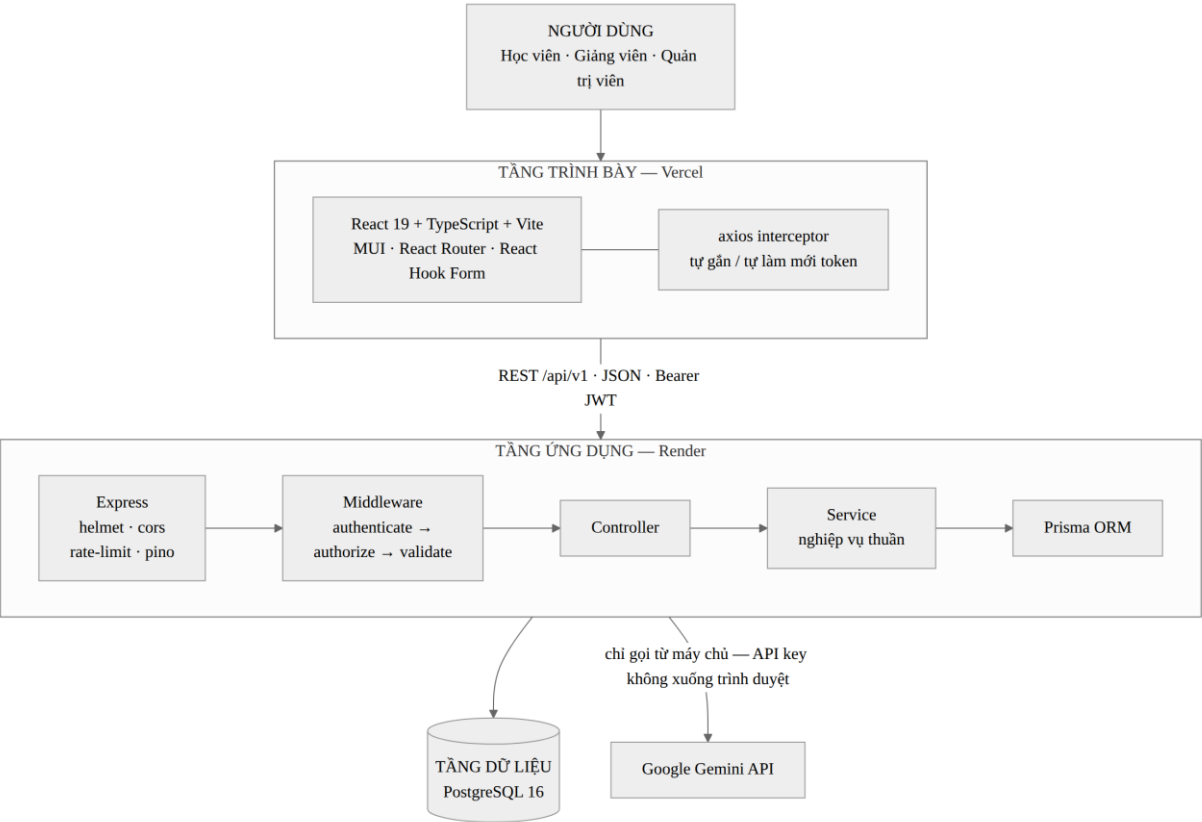
5 Yup

Một thư viện kiểm chứng cho cả hai đầu; loại bỏ trường lạ

6 Docker · Render · Vercel

Môi trường đồng nhất; triển khai lặp lại được bằng tệp cấu hình

Kiến trúc ba tầng



Tầng trình bày — Vercel

Ứng dụng một trang React; axios tự gắn và tự làm mới token.

Tầng ứng dụng — Render

Toàn bộ nghiệp vụ, quyền hạn và tính đúng đắn của dữ liệu.

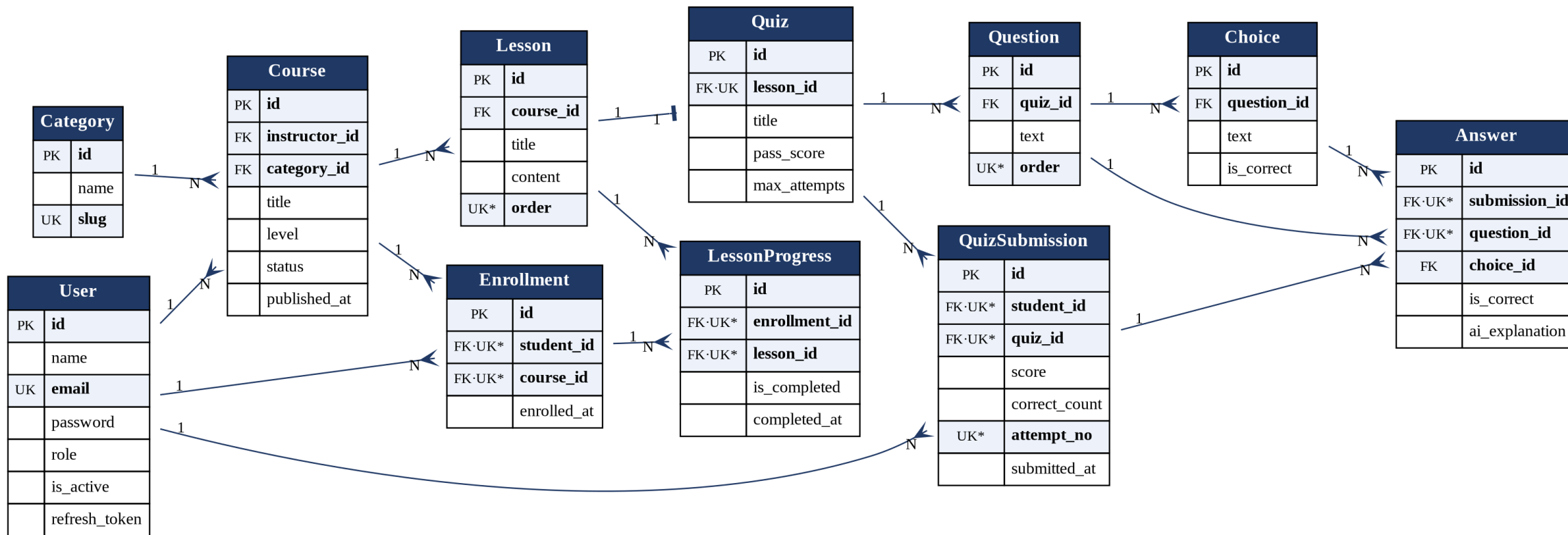
Tầng dữ liệu — PostgreSQL

Chín ràng buộc duy nhất đặt ngay tại cơ sở dữ liệu.

Gemini nằm ngoài hệ thống

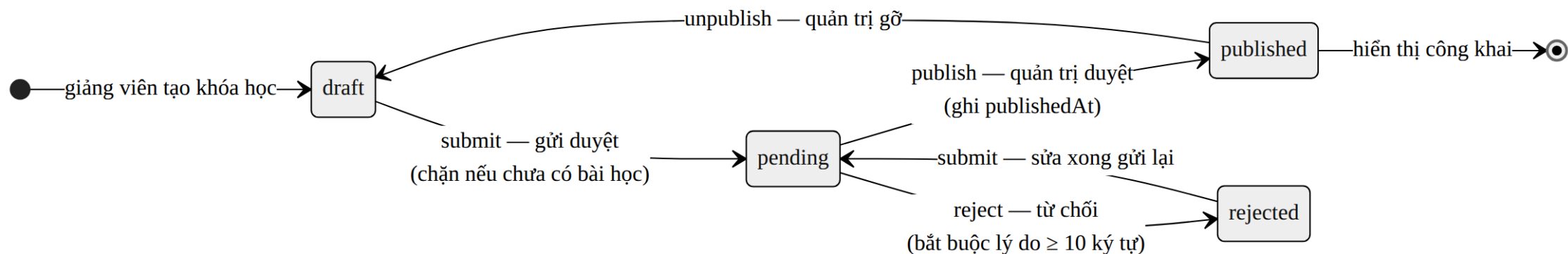
Chỉ tầng ứng dụng gọi; khóa API không bao giờ xuống trình duyệt.

Cơ sở dữ liệu: 11 bảng, ràng buộc đặt đúng tầng



- UNIQUE(lesson_id) → quan hệ 1–1 giữa bài học và quiz
- UNIQUE(course_id, order) → không hai bài trùng thứ tự
- UNIQUE(student_id, course_id) → không ghi danh trùng

Vòng đời khóa học là một máy trạng thái



Vì sao không dùng một cờ đúng/sai?

Một cờ `is_published` không diễn tả được trạng thái “đã gửi, đang chờ duyệt” — vốn chính là yêu cầu trọng tâm của vai trò quản trị viên.

Một bảng luật, hai nơi dùng chung

Giảng viên và quản trị viên đọc cùng một bảng chuyển trạng thái, nên không thể hiểu luật khác nhau. Đủ 16 tổ hợp đã được kiểm thử.

Đáp án đúng không bao giờ rời khỏi máy chủ

```
select: {  
  id: true, text: true, order: true,  
  // KHÔNG có isCorrect ở đây – cố ý  
  choices: {  
    select: { id: true, text: true }  
  },  
}
```

Không phải “ẩn khi hiển thị”

Cột isCorrect không được đọc lên khỏi cơ sở dữ liệu trong luồng lấy đề.

Được máy chứng minh

Bộ kiểm thử gọi HTTP thật rồi khẳng định chuỗi isCorrect không có trong phản hồi.

Kiểm tra kết quả, không kiểm tra ý định

Phép thử vẫn đúng sau mỗi lần sửa mã nguồn.

Chứng minh trực tiếp khi demo: mở tab Mạng của trình duyệt.

Chấm điểm chỉ diễn ra ở máy chủ

1

Trình duyệt gửi

chỉ [{questionId, choiceId}]

2

Middleware

xác thực · phân quyền · Yup

3

Máy chủ đọc đề

kèm cờ đáp án đúng — chỉ trong bộ nhớ

4

Hàm chấm thuần

gradeQuiz(questions, answers)

5

Lưu một giao dịch

lượt nộp + từng câu trả lời

Bắt 1 — mẫu số

Duyệt theo danh sách câu hỏi thật trong đề, không duyệt theo bài làm. Bỏ trống 3 câu không biến đề 5 câu thành đề 2 câu.

Bắt 2 — mã đáp án

Đáp án đã chọn phải thuộc đúng câu hỏi đó. Gửi mã đáp án của câu khác bị coi là bỏ trống.

Gửi kèm score = 100?

Trường lạ bị loại bỏ. Máy chủ vẫn chấm ra điểm thật — có phép kiểm thử riêng cho tình huống này.

Tích hợp AI: ba tính năng, một nguyên tắc

Sinh câu hỏi

Giảng viên

Chỉ trả bản nháp — AI không ghi vào cơ sở dữ liệu

Giải thích đáp án sai

Học viên

Lưu lại kết quả; lần thứ hai không gọi lại Gemini

Tóm tắt bài học

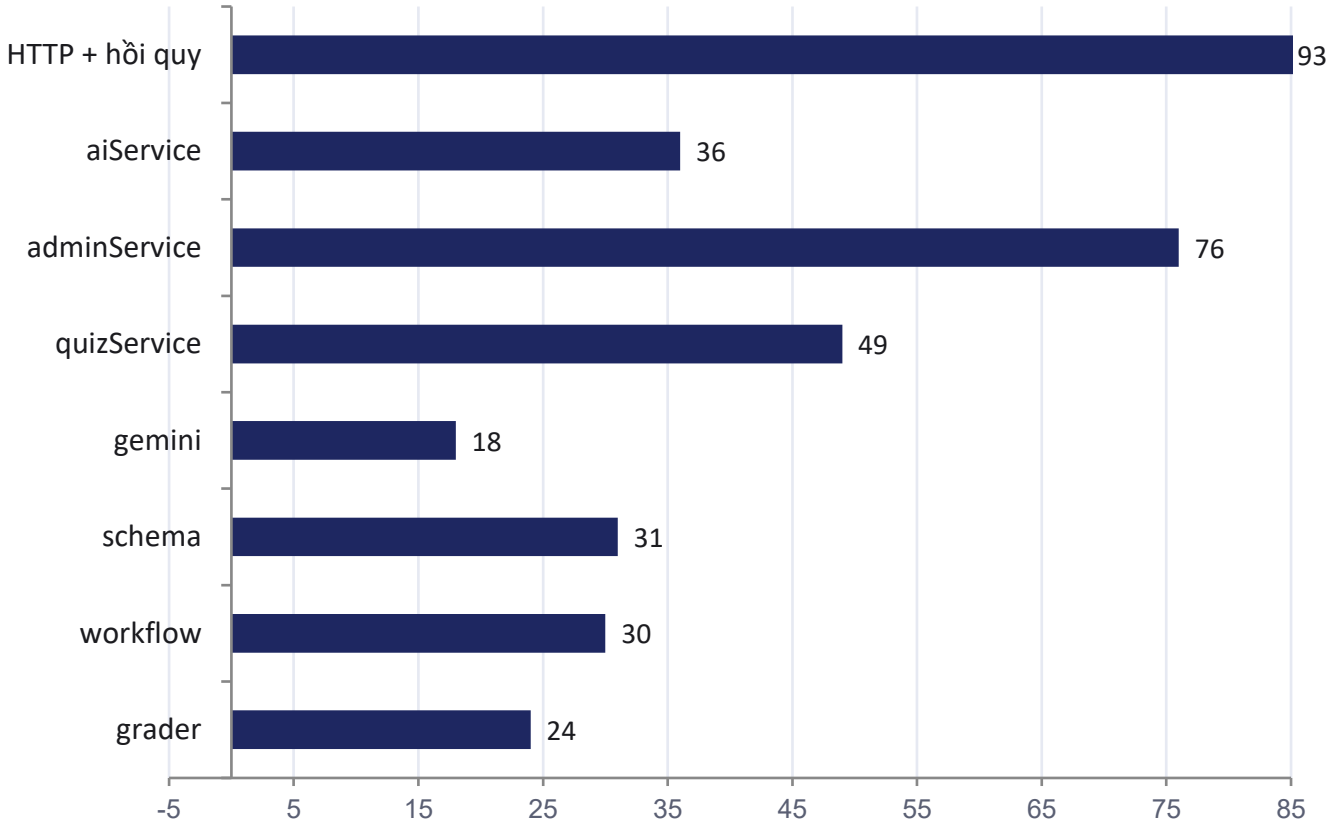
Học viên

3–5 ý, luôn kèm ghi chú nguồn gốc AI

Nguyên tắc: đầu ra của AI là dữ liệu không đáng tin

- Câu hỏi do Gemini sinh được kiểm chứng bằng đúng lược đồ dùng cho câu hỏi giảng viên gõ tay: đúng 4 đáp án, đúng 1 đáp án đúng.
- Ba đáp án hoặc hai đáp án đúng → chặn ở mã 422, không tới tay giảng viên.
- Thiếu khóa API → các nút AI bị làm mờ, phần còn lại của hệ thống vẫn chạy bình thường.

Kiểm thử tự động: 357 / 357 phép khẳng định



Ba lớp kiểm thử

Hàm thuần → nghiệp vụ → HTTP thật qua toàn bộ chuỗi middleware.

Không cần cơ sở dữ liệu

Prisma được thay bằng bản giả lập trong bộ nhớ — CI chạy xong trong khoảng một phút.

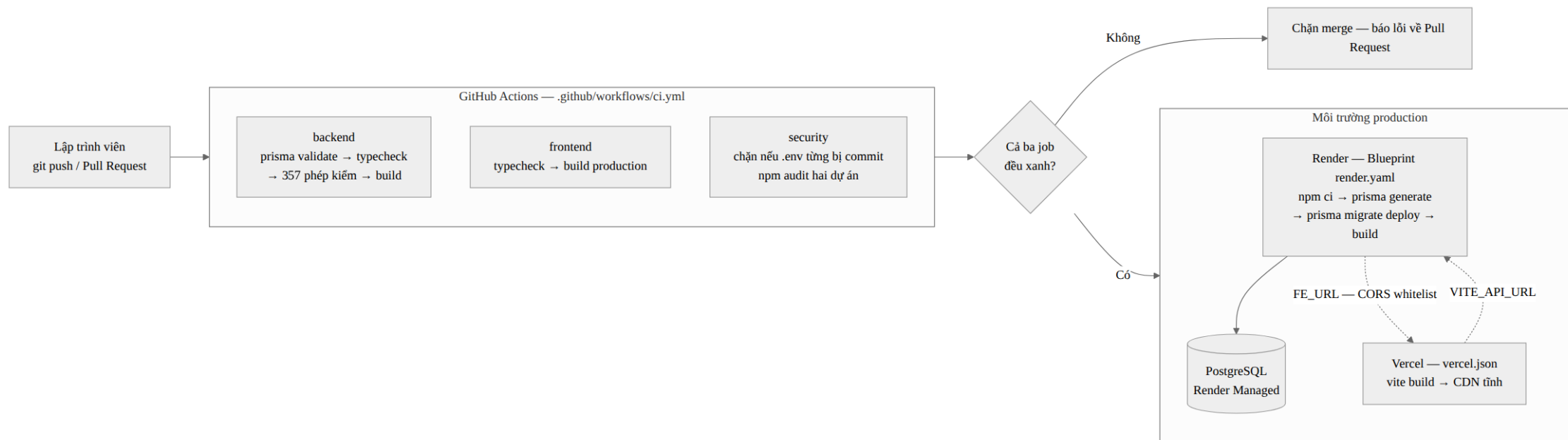
Bản giả lập tôn trọng select

Nếu không, phép thử “không rò rỉ đáp án” sẽ mất hết giá trị chứng minh.

Đã tìm ra lỗi thật

Luật mở khóa bài học từng cho phép trạng thái vô lý; kiểm thử phát hiện trước khi có người dùng.

Đóng gói, tích hợp liên tục và triển khai



Docker hai giai đoạn

Ảnh không chứa mã TypeScript, chạy bằng người dùng thường chứ không phải root.

Ba nhóm kiểm tra

Back-end · Front-end · Bảo mật (chặn nếu tệp .env từng bị đưa lên git).

Vercel (FE) + Render (BE)

Giao diện không bao giờ ngủ nên mở tức thì; máy chủ kịp thức dậy trong lúc người dùng đọc trang chủ.

Kịch bản demo — 8 phút

0–1	Khách	Duyệt và lọc danh sách khóa học
1–2	Học viên	Ghi danh, học bài 1, tiến độ tăng, bài 2 mở khóa
2–4	Học viên	Làm quiz, cố tình sai một câu, nộp, xem lại đáp án
4–5	Học viên	Bấm “Vì sao sai?”, rồi bấm lại — lần hai lấy từ bản đã lưu
5–6	Giảng viên	Gemini sinh 5 câu hỏi, sửa một câu rồi lưu
6–7	Quản trị	Duyệt khóa đang chờ — hiển thị công khai ngay
7–8	Kỹ thuật	Mở tab Mạng: phản hồi không chứa isCorrect · mở /health

Dự phòng: ping /health trước 30 phút để đánh thức dịch vụ; chuẩn bị ảnh chụp kết quả AI phòng khi hết hạn mức.

Kết quả đạt được

357/357

phép kiểm thử đạt

11

bảng dữ liệu, 9 ràng buộc duy nhất

46

điểm cuối API trên /api/v1

10.613

dòng mã nguồn TypeScript

Ba điểm có giá trị kỹ thuật cao nhất

- Ranh giới tin cậy vạch rõ và được máy chứng minh — khẳng định “đáp án không rời khỏi máy chủ” được kiểm chứng lại sau mỗi lần sửa mã.
- Ràng buộc toàn vẹn đặt ở tầng cơ sở dữ liệu, nên vẫn đúng khi có nhiều yêu cầu đồng thời.
- Quy tắc nghiệp vụ tách thành hàm thuần — kiểm thử được tức thì, và đã giúp phát hiện một lỗi lô-gic thật.

Hạn chế và hướng phát triển

Hạn chế — nêu trung thực

- Chưa đo hiệu năng dưới tải thật; kết luận về số truy vấn dựa trên phân tích.
- Chưa đo độ bao phủ kiểm thử bằng công cụ — bộ test thiết kế theo rủi ro nghiệp vụ.
- Chất lượng nội dung AI sinh chưa đánh giá định lượng; hệ thống chỉ bảo đảm cấu trúc hợp lệ.
- Bốn cột khóa ngoại chưa có chỉ mục — đã rà đủ 15 cột, ghi lại quyết định kèm bằng chứng thay vì thêm vội.

Hướng phát triển

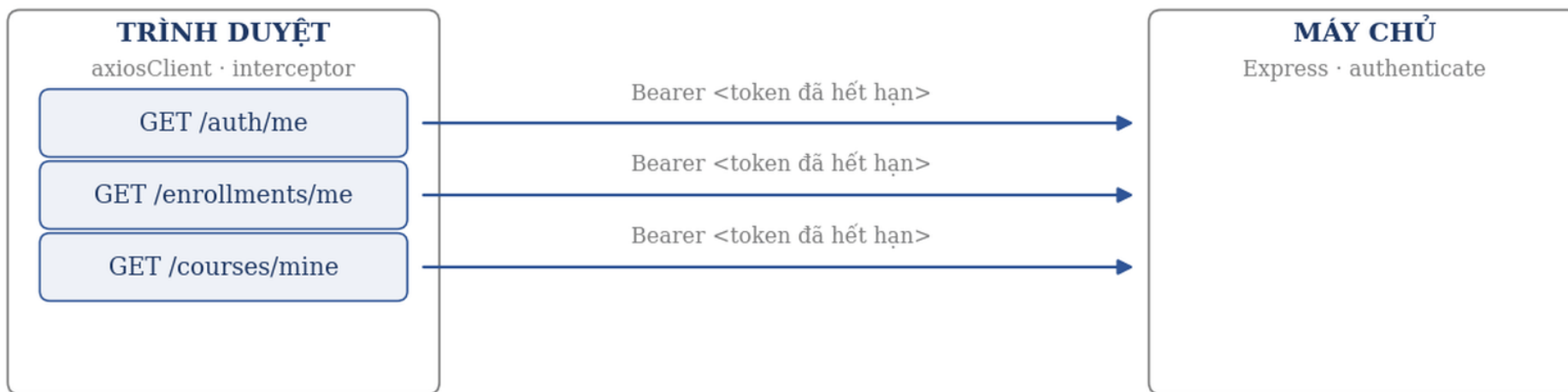
- Ngân hàng câu hỏi, sinh đề ngẫu nhiên mỗi lượt làm
- Chấm điểm có trọng số và giới hạn thời gian làm bài
- Nhật ký thao tác quản trị: ai duyệt, ai khóa, lúc nào
- Chứng chỉ hoàn thành khóa học dạng PDF
- Kiểm thử giao diện đầu–cuối tự động
- Gợi ý ôn tập dựa trên các câu đã trả lời sai

Xin cảm ơn quý thầy cô

Em xin lắng nghe nhận xét và câu hỏi.

Một lần làm mới cho cả nhóm yêu cầu

1. Ba yêu cầu cùng lúc, access token đã hết hạn



Vì sao phải có hàng đợi

Ba yêu cầu song song cùng nhận 401 sẽ gọi làm mới ba lần; token cũ bị thu hồi ngay lần đầu nên hai lần sau thất bại.

Một chờ và một hàng đợi

Cờ isRefreshing chặn lần gọi thứ hai; hàng đợi giữ các yêu cầu chờ rồi phát lại bằng token mới.

Không có vòng lặp 401

Cờ _retry chỉ cho mỗi yêu cầu thử lại đúng một lần; khi refresh token hết hiệu lực thì đăng xuất sạch.